

Functions III — Composition

Lecture 10

CS 1001

Kirk Duran

Today

- ▶ Extend the one-call model to programs with multiple active function calls.
- ▶ Represent active calls using separate execution environments organized by a call stack.
- ▶ Distinguish runtime execution environments from lexical scope and name visibility.
- ▶ Trace functions that call other functions, including nested call expressions.
- ▶ Use returned values to compose smaller computations into larger ones.
- ▶ Treat built-in and library functions through the same argument–return-value interface.

Key idea

Lecture 9 traced one active call. Today we keep unfinished callers active while additional calls begin.

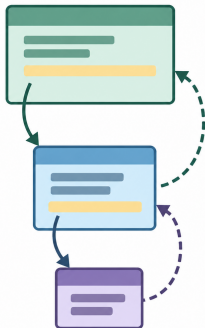
Part 1: From One Active Call to Multiple Calls

One Function Can Call Another Function

Program

```
def double(n: int) -> int:  
    return 2 * n  
  
def square_perimeter(length: int) ->  
int:  
    two_sides = double(length)  
    return two_sides + two_sides
```

- ▶ Calling `square_perimeter` begins one function call.
- ▶ Its body then begins a second call to `double`.



The Caller Is Suspended, Not Finished

Execution reaches

```
two_sides = double(length)
```

1. Evaluate `length` to obtain the argument value.
2. Begin a new call to `double`.
3. Suspend execution of `square_perimeter` at the call site.
4. Execute `double` until it returns.
5. Resume `square_perimeter` and complete the assignment.

Key idea

An unfinished caller retains its local bindings while the callee executes.

Active Calls Form a Stack

Definition

The **call stack** is the runtime ordering of currently active function calls, organized by caller–callee relationships.

While `double` executes

`double`
↑ current call
`square_perimeter`
`initial code`

- ▶ The most recently started unfinished call is current.
- ▶ Earlier callers remain beneath it.
- ▶ Returning removes the current call and resumes its immediate caller.

Each Active Call Has Its Own Execution Environment

double call

n -> 2

square_perimeter call

length -> 2

two_sides: not yet bound

- ▶ Each call has a distinct set of parameter and local-variable bindings.
- ▶ The outer call's bindings do not disappear when the inner call begins.
- ▶ The current frame changes as calls begin and return.

Key idea

Same program, multiple simultaneously active environments.

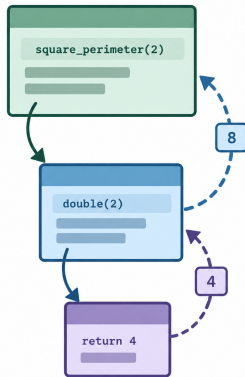
Returning Unwinds One Call at a Time

Inside double

```
return 2 * n  
2 * 2 -> 4  
return 4
```

Resume square_perimeter

```
two_sides = 4  
return 4 + 4  
return 8
```



Part 2: Execution Environments and Scope

Execution Environment and Scope Are Related but Different

Definition

An **execution environment** is a runtime mapping from names to values for one active call.

Runtime question

What bindings exist in this active call right now?

Definition

A **scope** determines where a name's binding is visible in program text.

Name-resolution question

Which binding does this occurrence of a name refer to?

The Same Spelling Can Refer to Different Bindings

Program fragment

```
x = 10  
  
def compute(value):  
    x = value + 1  
    return x
```

Module-level binding

x -> 10

During compute(4)

value -> 4
local x -> 5

Key idea

A variable name is not globally unique; its meaning depends on its scope and active environment.
A local assignment to x does not rebind a separate module-level x.

A Callee Does Not Inherit Its Caller's Local Variables

- ▶ A function's local scope is determined by its definition, not by whichever function happened to call it.
- ▶ Calling `adjust` from `compute` does not make `compute`'s local names automatically local to `adjust`.
- ▶ Data should cross the call boundary explicitly through arguments and returned values.

Preferred data flow

caller value → argument → parameter → computation → return value

Key idea

Call-stack position describes control flow; lexical scope determines name visibility.

Worked Scope Trace: Three Different Bindings Named x

Program

```
def adjust(x: int) -> int:
    y = x + 2
    return y * 3

def compute(value: int) -> int:
    x = value + 1
    answer = adjust(x)
    return answer

x = 10    result = compute(4)
```

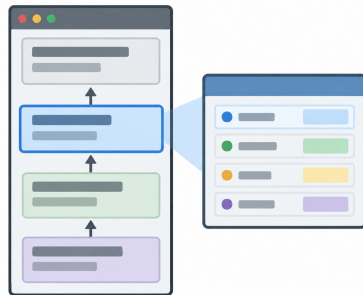
Environment	Relevant binding
module level	x -> 10
compute call	x -> 5
adjust call	parameter x -> 5

The Debugger Lets You Select Different Active Frames

- ▶ When stopped inside a nested call, the call stack lists active frames.
- ▶ Selecting a frame changes which local bindings the debugger displays.
- ▶ The program has not changed; you are inspecting a different active environment.

In class

While stopped inside `adjust`, inspect the `adjust`, `compute`, and module-level frames. Identify every binding named `x`.



Part 3: Function Composition

Composition Connects Returned Values to New Computations

Definition

Function composition uses the value produced by one function call as input to another computation.

Explicit data flow

```
step1 = add_one(4)
```

```
result = double(step1)
```

Equivalent nested form: `result = double(add_one(4))`

Data-flow interpretation

$4 \rightarrow \text{add_one} \rightarrow 5 \rightarrow \text{double} \rightarrow 10$

Nested Calls Encode the Same Data Flow More Compactly

With an intermediate name

```
step1 = add_one(4)  
result = double(step1)
```

Nested composition

```
result = double(add_one(4))
```

Key idea

The nested form removes an intermediate binding; it does not remove the intermediate value or the need to evaluate it.

Trace Nested Composition from the Inside Out

Expression

`double(add_one(4))`

1. Identify the outer callable `double`.
2. Evaluate its argument expression `add_one(4)`.
3. The `add_one` body executes and returns 5.
4. The outer call now has argument value 5.
5. The `double` body executes and returns 10.

Expression reduction

`double(add_one(4)) → double(5) → 10`

Composition Is Order-Sensitive

Double, then square

`square(double(3))`

`double(3) -> 6`

`square(6) -> 36`

Square, then double

`double(square(3))`

`square(3) -> 9`

`double(9) -> 18`

Key idea

Function composition is generally not commutative: changing the order can change the result.

A Call May Contain Multiple Function-Call Arguments

Example

```
result = add(square(3), square(4))
```

Values required before add can execute

square(3) -> 9

square(4) -> 16

then add(9, 16) -> 25

In class

How many distinct calls occur? During which call is the parameter name `n` bound to 3, and during which is it bound to 4?

Python Evaluates Argument Expressions from Left to Right

Expression

```
add(square(3), square(4))
```

1. Evaluate the first argument expression `square(3)` $\rightarrow$ 9.
2. Evaluate the second argument expression `square(4)` $\rightarrow$ 16.
3. Bind `x` $\rightarrow$ 9 and `y` $\rightarrow$ 16 for the call to `add`.
4. Execute `add` and return 25.

Tracing discipline

Do not merge the two calls to `square`; each call has its own parameter binding and lifetime.

Part 4: Tracing Functions That Call Functions

Parameter Bindings Must Be Kept Separate Across Calls

Program

```
def difference(a: int, b: int) -> int:
    return a - b

def distance_between(left: int, right: int) -> int:
    distance = difference(right, left)
    return distance
```

Call	Parameter bindings
distance_between(4, 10)	left -> 4, right -> 10
difference(right, left)	a -> 10, b -> 4

Full Trace: Follow Values Across Two Function Boundaries

1. `distance_between(4, 10)` creates `left -> 4, right -> 10`.
2. Evaluate arguments to `difference`: `right -> 10, left -> 4`.
3. The nested call creates `a -> 10, b -> 4`.
4. `difference` returns 6.
5. The suspended outer call binds `distance -> 6`.
6. `distance_between` returns 6 to its own caller.

Key idea

Record both the current frame and its bindings while tracing. Returned values travel one caller boundary at a time; they do not skip directly to the outermost code.

A Return Always Resumes the Immediate Caller

Call chain

initial code

↓

distance_between

↓

difference

Return chain

difference returns

↓

distance_between resumes

↓

then it may return to initial code

Common mistake

A nested function's returned value is first delivered to the expression that called it, not automatically to every outer caller.

Common Failure Modes with Nested Calls

Incorrect model	Correction
"The caller disappears while the callee runs."	The caller remains active and suspended on the stack.
"Same variable name means same variable."	Bindings belong to scopes and execution environments.
"A callee can see the caller's locals."	Call relationships do not create lexical scope relationships.
"Nested syntax executes from the outside inward."	Argument values must be produced before the outer function body can execute.
"Returning exits every active function."	A return completes only the current call.

Part 5: Library Functions and Abstraction Boundaries



Library Functions Participate in the Same Data-Flow Model

Example

```
from math import sqrt  
value = 25.0  
root = sqrt(value)
```

- ▶ Evaluate the argument expression `value` to obtain `25.0`.
- ▶ Call `sqrt` with that value.
- ▶ Receive the returned value `5.0`.
- ▶ Bind `root` -> `5.0`.

Key idea

The same argument–return–value model applies even when we do not inspect the function's implementation.

Capstone Example: User-Defined Functions Plus `sqrt`

Program

```
from math import sqrt
def square(n: float) -> float:
    return n * n
def magnitude(x: float, y: float) -> float:
    x_squared = square(x)
    y_squared = square(y)
    result = sqrt(x_squared + y_squared)
    return result
answer = magnitude(3, 4)
```

In class

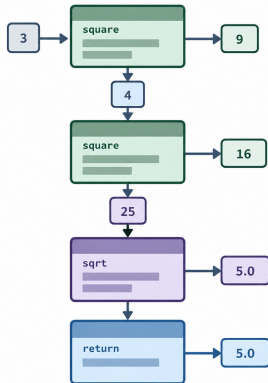
Predict every intermediate value and the call stack while the first call to `square` is executing.

Trace the Data Before Tracing the Implementation

Data flow

```
3 -> square -> 9
4 -> square -> 16
9 + 16 -> 25
25 -> sqrt -> 5.0
5.0 -> return -> answer
```

- ▶ First establish what values must flow between calls.
- ▶ Then inspect frames when a detailed execution trace is required.



Use an Abstraction Boundary for Trusted Library Functions

User-defined function under study

Trace inside when you need to understand:

- ▶ parameter bindings;
- ▶ local assignments;
- ▶ nested calls;
- ▶ returned values.

Trusted library function

Often reason from its documented interface:

- ▶ accepted arguments;
- ▶ documented behavior;
- ▶ returned value.

Key idea

Abstraction lets us choose the appropriate level of detail for the reasoning task.

Debugger Strategy: Step Into Selectively

- ▶ Use **Step Into** when the implementation of a user-defined call is part of the concept you are tracing.
- ▶ Use **Step Over** when a trusted call can be treated through its interface.
- ▶ Inspect the call stack whenever more than one user-defined call is active.
- ▶ Select different frames to compare their parameter and local-variable bindings.

Lab 3 discipline

Predict the stack and bindings first; then use the debugger to test the prediction.

Part 6: Decomposition and Program Design

Decomposition and Composition Are Complementary Ideas

Definition

Decomposition separates a larger computation into smaller functions with focused responsibilities.

Definition

Composition reconnects those functions by passing produced values into later computations.

Design pattern

large problem → smaller transformations → connected data flow

Worked Design: Separate Discount and Tax Calculations

Smaller transformations

```
def apply_discount(price, rate):  
    return price - price * rate  
  
def apply_tax(price, rate):  
    return price + price * rate
```

Composition

```
def final_price(price, discount,  
tax):  
    discounted =  
        apply_discount(price, discount)  
    return apply_tax(discounted, tax)
```

Key idea

Each function names one transformation; the outer function expresses how the transformations are connected.

Good Interfaces Make Composition Easier to Reason About

- ▶ Parameters should represent the data a computation genuinely requires.
- ▶ Returned values should expose the result needed by the caller.
- ▶ Avoid relying on unrelated external state when a value can be passed explicitly.
- ▶ Prefer functions whose names communicate the transformation they perform.

Mental contract

required input data → named computation → returned result

Why this matters for tracing

Explicit inputs and outputs make the path of data visible across function boundaries.

Practice: Trace a Composed Program

Program

```
def triple(n: int) -> int:
    return n * 3

def difference(a: int, b: int) -> int:
    return a - b

def compute(x: int, y: int) -> int:
    left = triple(x)
    right = triple(y)
    return difference(left, right)

answer = compute(5, 2)
```

In class

Predict: (1) every parameter binding, (2) every returned value, (3) the maximum call-stack depth, and (4) the final binding of answer.

Takeaways

- ▶ A function may begin another call before it has finished; unfinished callers remain active.
- ▶ The call stack records active calls, while each call has its own runtime execution environment.
- ▶ Scope determines which binding a name refers to; call-stack position does not create new lexical visibility.
- ▶ Returned values let functions be composed so that one computation supplies data to another.
- ▶ Nested argument expressions must produce values before the receiving function body can execute.
- ▶ Returning completes only the current call and resumes its immediate caller.
- ▶ Library functions can often be reasoned about through their documented input–output interface.
- ▶ Decomposition creates focused transformations; composition connects them into a larger program.

Key idea

Mental model: **values flow through calls; active calls keep separate bindings; return values reconnect the suspended computation.**